

RentChain

A blockchain-based rental house marketplace.

Project type: Mini Project — Blockchain Technology

Course: 6th Semester, BE Computer Science & Engineering

Institution: Visvesvaraya Technological University (VTU)

Tech stack:

Solidity 0.8.24

Hardhat 2

ethers.js v6

MetaMask

HTML / CSS / JS

1. Abstract

Traditional rental websites such as 99acres, NoBroker, and MagicBricks rely on a central server owned by a single company. The company controls the data, decides what gets shown, and brokers in the middle introduce extra cost and trust issues.

RentChain replaces that central server with an *Ethereum smart contract*. House owners list their properties on the blockchain. Tenants pay rent in cryptocurrency directly to the smart contract, which then credits the owner's account inside the contract. The owner can withdraw the collected rent at any time. Nobody — not even the developer — can secretly alter or delete a listing or steal the rent.

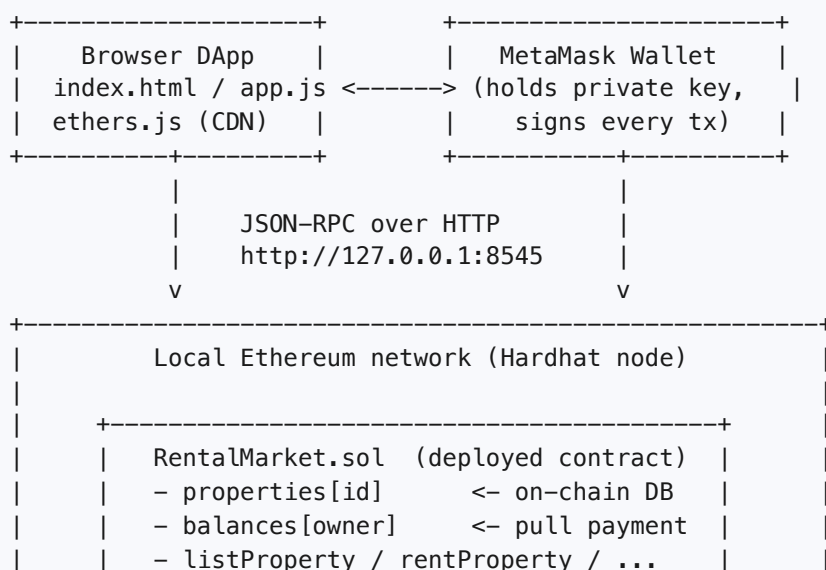
The project is intentionally scoped small so it can be demonstrated end-to-end in under two minutes. It illustrates the three core ideas of any blockchain course: **immutable on-chain storage**, **smart-contract logic**, and **trust-less peer-to-peer value transfer**.

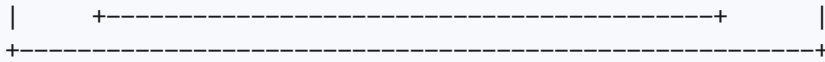
2. What is a blockchain (in 30 seconds)?

A blockchain is a shared, append-only database that is copied across thousands of computers around the world. Once data is written into a block, every copy of the chain has it; no single party can secretly change or remove it. **Ethereum** is a blockchain that adds *smart contracts* — small programs that live on the chain and run when called. Solidity is the language we write them in.

In this project we don't use the real Ethereum network — we use a **local Hardhat node**, which is a simulator running on the laptop. The simulator pretends to be Ethereum and gives us 20 fake test accounts, each pre-loaded with 10000 fake ETH. The same Solidity code would run unchanged on the real Ethereum mainnet.

3. System architecture





Smart contract — `RentalMarket.sol`

The brain of the system. Stores all properties and balances. Enforces all rules: only the owner can withdraw, you can't rent your own property, exact rent must be paid, etc.

Hardhat node

A local Ethereum simulator running on the laptop. Provides the JSON-RPC endpoint at `127.0.0.1:8545` and 20 funded test accounts.

MetaMask

Browser extension that holds the user's private key. Every time the contract is called, MetaMask pops up and asks the user to confirm and sign.

Frontend

Plain HTML, CSS, and JavaScript. Uses the **ethers.js** library (loaded from a CDN) to talk to the contract through MetaMask.

4. Smart-contract functions

Function	Who calls it	What it does
<code>listProperty(location, description, rentPerMonthWei)</code>	Owner	Stores a new property in <code>properties[id]</code> . The caller becomes the owner.
<code>rentProperty(id, months)</code> <code>payable</code>	Tenant	Tenant sends exact <code>rent × months</code> ETH along with the call. The contract records the tenant and the lease end date, and credits the owner's balance.
<code>endLease(id)</code>	Owner	After the lease end date passes, the owner ends the lease and the property becomes vacant.
<code>unlist(id)</code>	Owner	Removes a vacant property from the listing.
<code>withdraw()</code>	Owner	Owner pulls collected rent out into their own wallet (pull-payment pattern).
<code>getAllProperties()</code> view	Anyone	Returns the full list of properties — used by the frontend to render cards.

Why “pull payment” instead of forwarding rent automatically?

If the contract tried to send ETH to the owner inside `rentProperty()`, a malicious owner could write a contract that re-enters and drains the system (a re-entrancy attack), or simply reject the transfer to break future rentals. Storing the amount in `balances[owner]` and making the owner call `withdraw()` later is the textbook safe pattern.

5. The journey of a rent payment (step-by-step)

1. Tenant clicks **Rent** in the browser.
2. Frontend (`app.js`) calls `contract.rentProperty(id, months, { value })` using ethers.js.
3. Ethers.js asks MetaMask: “please sign this transaction”. MetaMask pops up and shows the gas fee and the ETH being sent.
4. User clicks **Confirm**. MetaMask signs the transaction with the user's private key and forwards it to the Hardhat node.
5. Hardhat node validates the transaction, runs the `rentProperty` code, updates contract storage, and includes the transaction in a new block.
6. The contract emits a `PropertyRented` event.

7. The frontend hears the confirmation, calls `getAllProperties()` again, and re-renders the card as “Rented until ...”.

6. How to run the project

You need three terminals open in the project folder.

Terminal 1 — start the local blockchain

```
npx hardhat node
```

Prints 20 funded test accounts. Leave this running.

Terminal 2 — deploy the contract

```
npx hardhat run scripts/deploy.js --network localhost
```

Outputs `RentalMarket deployed to: 0x...` and writes the address + ABI into `frontend/contract-info.js`.

Terminal 3 — serve the website

```
npx http-server frontend -p 5500 -c-1 -o
```

Opens `http://localhost:5500` in the browser.

One-time MetaMask setup

1. Networks → Add network manually → Name `Hardhat Local`, RPC URL `http://127.0.0.1:8545`, Chain ID `31337`, Currency `ETH`.
2. Account icon → Import account → paste a private key from Terminal 1.
3. Switch MetaMask to `Hardhat Local`.

7. Live demo flow (~ 90 seconds)

Action	What to say
Click Connect MetaMask	"I'm connecting as the <i>owner</i> of a property. The wallet address is my identity — no signup, no email."
List <i>BTM Layout, 2BHK</i> , rent 0.01 ETH	"The browser sends a transaction to the smart contract. MetaMask signs it. Once mined, the property is permanently on-chain."
Switch MetaMask to a second imported account	"Now I'm a different person — a <i>tenant</i> . Same website, same contract, different wallet."
Click Rent → confirm 0.01 ETH	"I'm sending 0.01 ETH directly to the contract. The contract automatically credits the owner and records me as the tenant."
Switch back to owner account → click Withdraw	"Back to the owner. I pull my earnings out — owner-to-tenant, peer-to-peer, enforced entirely by code."

8. Possible viva questions

Concept questions

Q. What is a blockchain?

A shared, append-only database replicated across many computers. Once data is in a block, no single party can secretly change it.

Q. What is a smart contract?

A program stored on the blockchain. It has its own storage and functions. It runs when someone sends a transaction calling one of its functions, and the result is recorded permanently in a new block.

Q. Why use blockchain for renting houses instead of a normal database?

Three reasons. (1) No central admin can secretly delete or alter listings. (2) Anyone can verify ownership and rent payments by looking at the chain. (3) The contract enforces the rules — the user doesn't have to trust a company.

Q. What is Ethereum?

A blockchain that, unlike Bitcoin, supports smart contracts. We write contracts in Solidity, compile them to EVM bytecode, and deploy them to the network.

Q. What is gas?

A fee paid in ETH to the network for executing a transaction. Bigger or more complex transactions consume more gas.

Q. What is MetaMask?

A browser-extension wallet that holds the user's private key, displays the account balance, and signs every transaction the website asks for. The website never sees the private key.

Code questions

Q. What is `msg.sender` ?

The Ethereum address of whoever called the function. Solidity's built-in identity check — we use it to verify ownership and to record who is renting.

Q. What does the `payable` keyword do?

Marks a function as one that can receive ETH along with the call. `rentProperty` is `payable` because the tenant pays rent at the same time as calling it.

Q. Why use the pull-payment pattern for `withdraw` ?

Sending ETH automatically inside `rentProperty` exposes the contract to re-entrancy attacks and stuck-funds DoS. Storing the amount in `balances[owner]` and letting the owner call `withdraw()` on their own is the standard secure pattern.

Q. What does `30 days` mean in Solidity?

A built-in time unit equal to `30 × 24 × 60 × 60` seconds. We use it to compute the lease end timestamp.

Q. What are events used for?

Events are logs the contract emits when state changes. The frontend listens for them so the UI can refresh after a transaction is mined.

Setup questions

Q. Is this real ETH? Did you spend money?

No. We're running on a local Hardhat node — a developer simulator. The 10000 ETH is fake test currency that only exists on the laptop.

Q. How would you deploy this on real Ethereum?

Add a `sepolia` entry in `hardhat.config.js` with an Infura/Alchemy RPC URL and a funded private key, then run the same deploy script with `--network sepolia`. The contract code does not change.

Q. What is Hardhat?

A development environment for Ethereum. It compiles Solidity, runs a local test blockchain, deploys contracts, and runs unit tests.

Q. What is ethers.js?

A JavaScript library that wraps Ethereum's JSON-RPC API. It connects to MetaMask, formats transactions, and decodes contract responses.

Tricky questions

Q. What happens if the rent amount is wrong?

The contract has `require(msg.value == total, "Send exact rent amount");` so the entire transaction reverts. No state changes, no ETH is taken.

Q. What stops the owner from renting their own property?

The line `require(p.owner != msg.sender, "Owner cannot rent own property");` in `rentProperty` .

Q. Can two tenants rent the same property at once?

No. `require(p.currentTenant == address(0) || block.timestamp >= p.leaseEnd, "Already rented");` blocks a second rental until the existing lease ends.

Q. How is the data structured on-chain?

A `mapping(uint256 => Property)` from property id to a struct, plus a `mapping(address => uint256)` for owner balances. Mappings are Solidity's key-value store.

9. Testing

The contract is covered by 8 unit tests written with Mocha + Chai (via `hardhat-toolbox`). They are run with:

```
npx hardhat test
```

#	Test	Branch covered
1	lists a property	happy path of <code>listProperty</code>
2	rejects listing with zero rent	<code>require(rentPerMonthWei > 0)</code>
3	rents and credits owner balance	happy path of <code>rentProperty</code>
4	rejects renting your own property	<code>require(p.owner != msg.sender)</code>
5	rejects wrong rent amount	<code>require(msg.value == total)</code>
6	rejects renting an already-rented property	active-lease check
7	lets owner withdraw earnings	happy path of <code>withdraw</code>
8	rejects withdraw when nothing to withdraw	<code>require(amount > 0)</code>

10. Advantages of RentChain

1. No middleman

Rent goes straight from tenant to owner through the contract. No broker fee, no platform commission (which is typically 1 month's rent on apps like NoBroker Pay or 99acres).

2. Tamper-proof listings

A central platform can secretly delete, hide, or reorder listings to favour paid promotions. On a blockchain, every listing is permanent and visible to everyone equally.

3. Verifiable payments

Every rent transaction is a public event on-chain. The tenant has a permanent on-chain receipt; the owner has permanent proof of payment. No "he said, she said" disputes.

4. No platform downtime

Centralized platforms can go down (or pivot, or shut down) and take all the data with them. The contract keeps running as long as Ethereum exists.

5. Code enforces the rules

Owners can't take the rent and refuse to mark the lease — the contract did that automatically. Tenants can't double-rent the same flat — the contract blocks it.

6. Identity is just a wallet

No signup, no email, no OTP. The wallet address is the identity. Useful for users who want privacy.

7. Global reach

Anyone with MetaMask can interact, regardless of country or bank. ETH transfers don't care about borders or banking hours.

8. Open and auditable

The contract source code is public. Anyone can verify the rules. Compare this to closed-source rental apps where pricing logic is hidden.

11. Comparison with current market apps

Feature	NoBroker	99acres	MagicBricks	RentChain
Where listings are stored	Their server	Their server	Their server	On Ethereum, public
Can the platform secretly hide a	Yes	Yes	Yes	No

listing?				
Broker commission	Zero (claim) but premium fee for direct contact	Up to 1 month's rent	Up to 1 month's rent	Zero. Only gas fee (~₹5).
Listing fee for owners	Free / Premium plans	Premium plans (paid)	Premium plans (paid)	Free. Only one-time gas fee.
Rent payment processing	Through bank / NoBroker Pay	Outside the platform	Outside the platform	On-chain in ETH (instant)
Identity / KYC	Phone OTP + KYC	Email + phone	Email + phone	Wallet address (anonymous)
Disputes settled by	Customer support team	Customer support team	Customer support team	Smart-contract logic
Server downtime risk	Yes	Yes	Yes	None as long as Ethereum runs
Open source	No	No	No	Yes
Available 24x7 globally	India only	India only	India only	Anywhere in the world

Honest trade-offs (mention these too — examiners like balanced answers):

- RentChain users need ETH and a wallet, which is a bigger learning curve than just downloading the NoBroker app.
- Gas fees on real Ethereum can spike during peak times. Solution: deploy on a Layer-2 chain like *Polygon* where fees are typically a fraction of a rupee.
- If the user loses their private key, the wallet (and any rent collected) is lost. Centralized apps support password reset.
- Real-world rent disputes (broken pipes, deposit refunds) still need a human; the contract only enforces the digital rules.

12. Improvements / Future scope

The current build is intentionally minimal. The following are concrete, demonstrable extensions that can be added without redesigning the system:

Improvement	What it adds	Why it matters
Security deposit + escrow	Tenant pays a refundable deposit at rent time; the contract holds it until the lease ends, then refunds or forfeits based on owner action.	Mirrors how real-world rental agreements work and removes deposit-refund disputes entirely.
IPFS image storage	Property photos uploaded to IPFS; only the content hash is stored on-chain.	Listings become visual without making the contract storage expensive.
Reputation / rating system	After a lease ends, both sides can rate each other. Ratings stored on-chain against the wallet address.	Replaces the “trust score” that centralized platforms calculate.
Stablecoin rent (e.g. USDC)	Pay rent in a USD-pegged stablecoin instead of volatile ETH.	Removes price-volatility risk so rent is a predictable amount each month.
Layer-2 deployment	Deploy the same contract to Polygon or Arbitrum.	Gas fees drop from \$0.30 to fractions of a cent, and confirmation is near-instant.
Auto-recurring rent	A scheduled job (e.g. Chainlink Automation) calls the contract monthly to deduct rent from the tenant.	True “set-and-forget” rent — matches how a standing instruction with a bank works.
KYC layer	Optional verified-identity NFT linked to the wallet, issued after KYC.	Lets owners restrict rentals to verified tenants without giving up the on-chain trust model.
Dispute arbitration	A neutral third-party arbitrator (or DAO) can be specified per lease to resolve real-world disputes.	Bridges the gap between code-enforced rules and physical-world problems.
Public testnet deployment	Deploy the same contract to Sepolia or Polygon Mumbai.	Gives a shareable URL anyone in the world can try, no Hardhat setup required.

13. Glossary (quick reference)

Term	Meaning
------	---------

Blockchain	An append-only ledger replicated across many machines.
Ethereum	A blockchain platform that supports smart contracts.
Smart contract	A program that lives on the blockchain.
Solidity	The programming language used to write Ethereum smart contracts.
EVM	Ethereum Virtual Machine — the runtime that executes smart contracts.
Wei	The smallest unit of ETH. 1 ETH = 10^{18} wei.
Gas	The fee paid to run a transaction, measured in ETH.
Wallet	Software that holds a private key and signs transactions (e.g. MetaMask).
DApp	Decentralized application — a frontend backed by a smart contract.
Hardhat	A Node.js development framework for Ethereum.
ABI	Application Binary Interface — describes how to call a contract's functions from outside.
Pull payment	A pattern where the recipient withdraws funds themselves instead of having them auto-sent.